Re:Life Health Website - Implementation Changes Document

Document Version: 1.0
Created: April 16, 2026
Target Audience: Development Team
Status: Ready for Implementation

Overview

This document outlines the specific implementation changes required for the Re:Life Health website revamp. The current prototype serves as the foundation, and this document details all modifications needed for the four primary pages: Home, Products, Services, and Application. The Wellness Center, About Us, and Contact Us pages will be updated in a later phase.

Note: Color scheme and design theme remain unchanged from the prototype. All changes are structural, functional, or content-based.

File Structure & Setup

Current Project Structure

```
client/src/
├── pages/
│   ├── Home.tsx ........................ [REQUIRES CHANGES]
│   ├── Products.tsx ................... [REQUIRES CHANGES]
│   ├── Services.tsx ................... [REQUIRES CHANGES]
│   ├── Application.tsx ................ [REQUIRES CHANGES]
│   ├── WellnessCenter.tsx ............ [NO CHANGES - PHASE 2]
│   ├── AboutUs.tsx .................... [NO CHANGES - PHASE 2]
│   ├── ContactUs.tsx .................. [NO CHANGES - PHASE 2]
│   └── NotFound.tsx
├── components/
│   ├── Navigation.tsx ................. [STABLE]
│   └── ui/ ............................ [STABLE]
├── App.tsx ............................ [STABLE]
├── index.css .......................... [STABLE - Colors/Theme Locked]
└── main.tsx ........................... [STABLE]
```

Key Files Not Requiring Changes

- `client/src/index.css` - Design tokens and color palette are finalized
- `client/src/App.tsx` - Routing structure is complete
- `client/src/components/Navigation.tsx` - Navigation is finalized
- All shadcn/ui components - Component library is stable

Page-by-Page Implementation Changes

1. HOME PAGE (`client/src/pages/Home.tsx`)

Current Status: Functional prototype with placeholder content
Required Changes: Content refinement, structure optimization, and integration preparation

1.1 Hero Section Modifications

Current Implementation:

```
<h1 className="text-4xl md:text-5xl font-bold text-foreground leading-tight">
  Your Mental Health & Wellness Journey Starts Here
</h1>
```

Changes Required:

- Review and finalize headline copy with marketing team
- Ensure headline resonates with target audience
- Consider A/B testing variants if needed
- Subheadline should clearly communicate unique value proposition

Implementation Notes:

- Keep existing Tailwind classes for responsive sizing
- Maintain font hierarchy (Poppins 700 for h1)
- No design changes - content only

1.2 Feature Cards Section

Current Implementation: 4-column grid with hardcoded features (Products, Programs, Wellness Center, App)

Changes Required:

- Verify feature card order aligns with business priorities
- Ensure icons (Heart, Leaf, Users, Pill) match intended meanings
- Consider if all 4 features are essential or if prioritization is needed
- Add descriptions that clearly explain each feature's value

Implementation Notes:

- Cards use `Link` component for navigation - verify all routes are correct
- Hover effects are CSS-based - no changes needed

- Grid layout is responsive - mobile view shows 1 column automatically

1.3 Call-to-Action Section

Current Implementation: Generic CTA with “Get Started Now” button

Changes Required:

- Finalize CTA copy and messaging
- Verify button action (currently links to #register anchor)
- Consider if CTA should link to specific page or registration flow
- Ensure CTA aligns with conversion goals

Implementation Notes:

- Button styling uses primary color - no changes needed
- Consider adding secondary CTA if needed
- Track CTA click-through rates for optimization

1.4 Footer Section

Current Implementation: Simple footer with links and copyright

Changes Required:

- Add social media links (Instagram, Facebook, YouTube, Twitter, LinkedIn)
- Include company address and contact information
- Add newsletter subscription option (optional)
- Ensure all footer links are correct

Implementation Notes:

- Footer uses dark background (`bg-foreground`) - no changes needed
- Link structure should match navigation
- Consider adding footer CTA for app download

---

2. PRODUCTS PAGE (`client/src/pages/Products.tsx`)

Current Status: Functional prototype with 6 sample products
Required Changes: Product data integration, filtering, cart functionality, and payment gateway setup

2.1 Product Grid & Data Management

Current Implementation: Hardcoded array of 6 products with sample data

Changes Required:

- Replace hardcoded product data with API calls to backend
- Implement product filtering by category
- Add search functionality
- Implement pagination if product count exceeds 12 per page
- Add sorting options (price, rating, newest)

Implementation Code Structure:

```
// Replace hardcoded products array with API call
const [products, setProducts] = useState([]);
const [loading, setLoading] = useState(true);

useEffect(() => {
  fetchProducts().then(data => {
    setProducts(data);
    setLoading(false);
  });
}, []);

// Add filter state
const [selectedCategory, setSelectedCategory] = useState('all');
const [sortBy, setSortBy] = useState('featured');
```

Backend API Requirements:

- `GET /api/products` - Fetch all products with optional filters
- `GET /api/products/categories` - Fetch available categories

- Response format should include: id, name, category, price, rating, reviews, description, features, image_url

2.2 Product Card Enhancements

Current Implementation: Static card layout with placeholder image gradient

Changes Required:

- Replace gradient placeholder with actual product images from CDN
- Implement image lazy loading for performance
- Add product availability status (in stock/out of stock)
- Add product SKU or product code
- Implement quick view modal (optional)

Implementation Notes:

- Image URL should come from API response
- Use `<img>` tag with `loading="lazy"` attribute
- Implement fallback for missing images
- Consider WebP format with PNG fallback

2.3 Add to Cart Functionality

Current Implementation: Button with no functionality

Changes Required:

- Implement shopping cart state management (Context API or Redux)
- Add quantity selector to product card or modal
- Implement add to cart with user feedback (toast notification)
- Track cart items in localStorage for persistence
- Add cart icon to navigation with item count

Implementation Code Structure:

```
const handleAddToCart = (product, quantity) => {
  addToCart(product, quantity);
  showToast(`${product.name} added to cart!`);
};

// Cart Context
const [cartItems, setCartItems] = useState([]);
const addToCart = (product, quantity) => {
  // Add logic to update cart
};
```

Integration Points:

- Shopping cart should persist across page navigation
- Cart should be accessible from navigation bar
- Implement cart summary page (future phase)
- Prepare for checkout flow integration

2.4 Product Reviews & Ratings

Current Implementation: Static 5-star rating with review count

Changes Required:

- Fetch rating data from backend API
- Implement review display (optional - can be phase 2)
- Add review submission form (optional - can be phase 2)
- Implement rating filter

Backend API Requirements:

- `GET /api/products/:id/reviews` - Fetch product reviews
- `POST /api/products/:id/reviews` - Submit new review (requires authentication)

2.5 Product Categories & Filtering

Current Implementation: No filtering, all products displayed

Changes Required:

- Add category filter sidebar or dropdown
- Implement multi-select filtering
- Add filter reset button
- Display active filters

Implementation Notes:

- Categories: Sleep Wellness, Relaxation, Complete Care, Mindfulness, Herbal, Ambiance
- Filter should update URL query parameters for bookmarking
- Maintain responsive design on mobile (collapse filters into drawer)

---

3. SERVICES PAGE (`client/src/pages/Services.tsx`)

Current Status: Functional prototype with sample services and events
Required Changes: Event data integration, registration system, and calendar functionality

3.1 Services Overview Section

Current Implementation: 4-column grid with hardcoded services

Changes Required:

- Fetch service data from backend API
- Add service filtering by type
- Implement service detail modal or dedicated page
- Add service registration capability

Implementation Code Structure:

```
const [services, setServices] = useState([]);

useEffect(() => {
  fetchServices().then(data => {
    setServices(data);
  });
}, []);
```

Backend API Requirements:

- `GET /api/services` - Fetch all services
- Response format: id, title, description, icon, schedule, participants, duration, category

3.2 Upcoming Events Management

Current Implementation: 3 hardcoded events with static data

Changes Required:

- Replace hardcoded events with API calls
- Implement event filtering by date, location, type
- Add calendar view for events (optional - can be phase 2)
- Implement real-time spot availability updates
- Add event registration system

Implementation Code Structure:

```
const [events, setEvents] = useState([]);
const [selectedDate, setSelectedDate] = useState(new Date());

useEffect(() => {
  fetchEvents(selectedDate).then(data => {
    setEvents(data);
  });
}, [selectedDate]);

const handleRegister = async (eventId) => {
  const response = await registerForEvent(eventId);
  if (response.success) {
    showToast('Successfully registered for event!');
  }
};
```

Backend API Requirements:

- `GET /api/events` - Fetch events with optional date filter
- `POST /api/events/:id/register` - Register user for event
- `GET /api/events/:id` - Get event details
- Response format: id, title, date, time, location, description, availableSpots, totalCapacity, category

3.3 Event Registration System

Current Implementation: "Register Now" button with no functionality

Changes Required:

- Implement registration form modal
- Collect user information (name, email, phone, dietary restrictions, etc.)
- Add form validation
- Send confirmation email to user
- Update available spots in real-time
- Prevent double registration

Implementation Notes:

- Registration requires user authentication (check if user is logged in)
- If not logged in, redirect to login/signup page
- Store registration data in database
- Send confirmation email with event details
- Implement registration cancellation option

3.4 Event Types & Categorization

Current Implementation: 4 service types listed separately

Changes Required:

- Consolidate services and events into unified data structure
- Implement event type filtering (Seminar, Workshop, Support Group, CSR Program)
- Add event difficulty/level indicator (Beginner, Intermediate, Advanced)
- Add prerequisite information if applicable

Implementation Notes:

- Event types should be managed in backend
- Filter should be multi-select capable
- Display active filters to user

3.5 Event Details & Information

Current Implementation: Limited information display

Changes Required:

- Add facilitator/speaker information
- Add event agenda or schedule
- Add location map integration (Google Maps for physical events)
- Add event description and learning outcomes
- Add cancellation policy
- Add refund information

Implementation Notes:

- Use existing Map component for location display
- Add event materials download option (post-event)
- Implement event reminder notifications

---

4. APPLICATION PAGE (`client/src/pages/Application.tsx`)

Current Status: Functional prototype with app feature showcase
Required Changes: App links integration, mobile app tracking, and feature updates

4.1 Web App Launch Button

Current Implementation: Button with no action (placeholder)

Changes Required:

- Update button to link to actual web app URL
- Implement authentication check (redirect to login if not authenticated)
- Add tracking for button clicks
- Consider adding feature tour for first-time users

Implementation Code Structure:

```
const handleLaunchWebApp = () => {
  // Track event
  trackEvent('web_app_launch_clicked');

  // Redirect to web app
  if (isAuthenticated) {
    window.location.href = '/webapp'; // or external URL
  } else {
    window.location.href = '/login';
  }
};
```

Integration Points:

- Verify web app URL is correct
- Ensure authentication tokens are passed correctly
- Test cross-domain authentication if web app is on different domain

4.2 Mobile App Notification System

Current Implementation: "Notify Me When Available" button with no functionality

Changes Required:

- Implement email capture form
- Store emails in database for mobile app launch notification
- Create automated email campaign for app launch
- Add app store links when app is available
- Track notification signup metrics

Implementation Code Structure:

```
const [email, setEmail] = useState('');
const [loading, setLoading] = useState(false);

const handleNotifyMe = async () => {
  setLoading(true);
  const response = await subscribeToMobileAppNotification(email);
  if (response.success) {
    showToast('You\'ll be notified when the app launches!');
    setEmail('');
  }
  setLoading(false);
};
```

Backend API Requirements:

- `POST /api/mobile-app/notify` - Subscribe to mobile app notifications
- Response format: success, message

4.3 Web App Features List

Current Implementation: Hardcoded list of 6 features

Changes Required:

- Verify features are accurate and up-to-date
- Add feature icons or illustrations
- Consider adding feature descriptions or tooltips
- Update features as new functionality is added
- Highlight most important features

Implementation Notes:

- Features should match actual web app capabilities
- Consider organizing features by category (Core, Analytics, Community)
- Add “Coming Soon” badges for upcoming features

4.4 Mobile App Features List

Current Implementation: Hardcoded list of 6 planned features

Changes Required:

- Confirm all features are planned for mobile app
- Add estimated release timeline
- Update features as development progresses
- Consider adding feature request form

Implementation Notes:

- Features should differentiate mobile app from web app
- Focus on mobile-specific capabilities (offline access, notifications, biometrics)
- Keep features realistic and achievable

4.5 Features Comparison Table

Current Implementation: Static table with hardcoded data

Changes Required:

- Verify all features and availability are correct

- Update table as features are added/removed
- Consider making table interactive (expandable rows, tooltips)
- Add feature descriptions on hover

Implementation Code Structure:

```
const features = [
  { name: 'AI Chatbot Support', web: true, mobile: true },
  { name: 'Appointment Booking', web: true, mobile: true },
  // ... more features
];

// Render dynamic table
{features.map((feature) => (
  <tr key={feature.name}>
    <td>{feature.name}</td>
    <td>{feature.web ? <Check /> : '—'}</td>
    <td>{feature.mobile ? <Check /> : '—'}</td>
  </tr>
))}
```

4.6 App Download Links

Current Implementation: Placeholder buttons with no action

Changes Required:

- Add App Store link (iOS) when app is available
- Add Google Play link (Android) when app is available
- Implement platform detection (redirect to correct store)
- Add app version information
- Track download clicks

Implementation Notes:

- Links should open in new tab
- Consider adding QR codes for mobile app download
- Add app rating/reviews display (optional)

Cross-Page Implementation Requirements

Authentication & User Management

Current Status: No authentication implemented
Required Changes:

1. **Login System**:
   - Implement login page at `/login`
   - Add authentication context/state management
   - Store authentication token (JWT) in localStorage
   - Implement token refresh mechanism
   - Add logout functionality
2. **Register Now Button**:
   - Update all "Register Now" buttons to redirect to login/signup
   - If user is authenticated, redirect to webapp
   - If not authenticated, show login/signup form
3. **Protected Routes**:
   - Implement route protection for authenticated pages
   - Redirect unauthenticated users to login

API Integration Framework

Current Status: No API calls implemented
Required Changes:

1. **API Client Setup**:
   - Create API client utility (`client/src/lib/api.ts`)
   - Implement error handling and retry logic
   - Add request/response interceptors for authentication
   - Implement loading and error states

2. **API Endpoints**:
    - Products: GET /api/products, GET /api/products/:id
    - Services: GET /api/services, GET /api/services/:id
    - Events: GET /api/events, POST /api/events/:id/register
    - Contact: POST /api/contact
    - Users: POST /api/auth/login, POST /api/auth/register
3. **Error Handling**:
    - Implement global error boundary
    - Add user-friendly error messages
    - Log errors for debugging
    - Implement retry mechanism for failed requests

State Management

Current Status: Component-level state only
Required Changes:

1. **Global State**:
    - User authentication state
    - Shopping cart state
    - User preferences/settings
    - Notification state
2. **State Management Solution**:
    - Consider using React Context API (lightweight)
    - Or implement Redux (if complex state needed)
    - Implement localStorage persistence for cart

Form Handling & Validation

Current Status: Contact form implemented with basic validation
Required Changes:

1. **Form Libraries**:
    - Consider using react-hook-form (already in dependencies)
    - Implement Zod for schema validation (already in dependencies)
    - Add field-level validation
    - Implement async validation (email uniqueness, etc.)
2. **Forms to Implement**:
    - Event registration form
    - Mobile app notification signup
    - Product review form (phase 2)
    - Contact form enhancements

Performance Optimization

Current Status: Basic optimization in place
Required Changes:

1. **Code Splitting**:
    - Implement lazy loading for pages
    - Use React.lazy() for route-based code splitting
    - Implement Suspense boundaries
2. **Image Optimization**:
    - Use next-gen formats (WebP with fallback)
    - Implement responsive images with srcset
    - Add image lazy loading
    - Compress images before upload
3. **Caching Strategy**:
    - Implement HTTP caching headers
    - Use service workers for offline support (optional)
    - Cache API responses appropriately

Testing Checklist

Functional Testing

- [ ] All navigation links work correctly
- [ ] Products load from API correctly
- [ ] Services and events display properly
- [ ] Event registration works end-to-end
- [ ] Add to cart functionality works
- [ ] Forms validate and submit correctly
- [ ] Authentication flow works
- [ ] Logout functionality works
- [ ] Protected routes redirect unauthenticated users

Integration Testing

- [ ] API calls return correct data
- [ ] Error handling works for failed API calls
- [ ] Authentication tokens persist correctly
- [ ] Cart persists across page navigation
- [ ] Forms send data to backend correctly

Performance Testing

- [ ] Page load time < 3 seconds
- [ ] Lighthouse score > 90
- [ ] Images load quickly
- [ ] API calls complete within acceptable time
- [ ] No memory leaks in browser

Responsive Design Testing

- [ ] Mobile view (320px - 480px)
- [ ] Tablet view (768px - 1024px)
- [ ] Desktop view (1280px+)
- [ ] All interactive elements work on touch devices
- [ ] Navigation menu works on mobile

Cross-Browser Testing

- [ ] Chrome/Edge (latest)
- [ ] Firefox (latest)
- [ ] Safari (latest)
- [ ] Mobile browsers (iOS Safari, Chrome Mobile)

Deployment & Launch Checklist

Pre-Deployment

- [ ] All changes implemented and tested
- [ ] Code review completed
- [ ] Performance optimized
- [ ] Security review completed
- [ ] SEO meta tags added
- [ ] Analytics tracking implemented
- [ ] Error monitoring configured

Deployment

- [ ] Deploy to staging environment
- [ ] Test all functionality in staging

- [ ] Deploy to production
- [ ] Monitor error logs
- [ ] Monitor performance metrics
- [ ] Monitor user analytics

Post-Deployment

- [ ] Verify all pages load correctly
- [ ] Test user flows end-to-end
- [ ] Monitor for errors and issues
- [ ] Gather user feedback
- [ ] Plan for phase 2 updates

Phase 2 Updates (Future)

The following pages will be updated in Phase 2:

1. **Wellness Center Page** (`/wellness-center`)
    - Integrate article management system
    - Implement search functionality
    - Add article categories and filtering
    - Implement user comments/discussions
2. **About Us Page** (`/about-us`)
    - Update team information
    - Add team member profiles
    - Implement team filtering
    - Add company timeline
3. **Contact Us Page** (`/contact-us`)
    - Enhance contact form with more fields

- Implement form submission tracking
- Add live chat support (optional)
- Implement ticket system

Developer Notes & Tips

Best Practices

1. **Component Reusability**: Extract common patterns into reusable components
2. **Error Handling**: Always handle API errors gracefully with user-friendly messages
3. **Loading States**: Show loading indicators for all async operations
4. **Accessibility**: Maintain WCAG 2.1 AA compliance throughout
5. **Performance**: Monitor bundle size and optimize as needed

Common Pitfalls to Avoid

1. **Hardcoded Data**: Replace all sample data with API calls
2. **Missing Error Boundaries**: Wrap components that might error
3. **Unhandled Promises**: Always handle async operations with try/catch
4. **Missing Loading States**: Show feedback for all async operations
5. **Broken Links**: Verify all navigation links are correct

Useful Commands

```
# Development
pnpm dev

# Type checking
pnpm check

# Build for production
pnpm build

# Preview production build
pnpm preview

# Format code
pnpm format
```

Support & Questions

For questions or clarifications about these implementation changes, refer to:

1. **Design System**: See `client/src/index.css` for design tokens
2. **Component Library**: Check `client/src/components/ui/` for available components
3. **API Documentation**: Backend team should provide API documentation
4. **Design Specification**: See `DEVELOPER_SPECIFICATION.md` for detailed design guidelines

---

Document History

Version	Date	Changes
1.0	April 16, 2026	Initial implementation guide for Home, Products, Services, and Application pages

---

Document Owner: Manus AI
Last Updated: April 16, 2026
Next Review: Upon Phase 2 completion